

Cálculo Paralelo do Conjunto de Mandelbrot

Versão Híbrida MPI + OpenMP

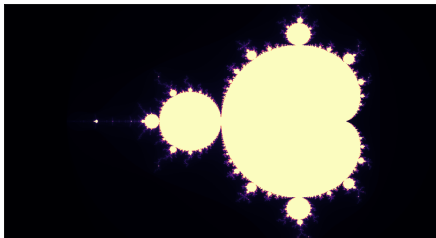
Bernardo Luiz Padilha Zamin • João Pedro Sostruznik Sotero da Cunha

Problema: conjunto de Mandelbrot (7680×4320) — cada pixel repete $z = z^2 + c$ até escapar. Pixels **independentes**, mas de **custo irregular**.

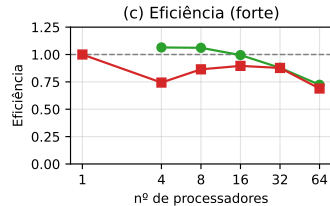
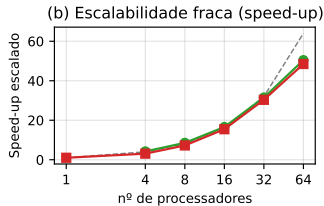
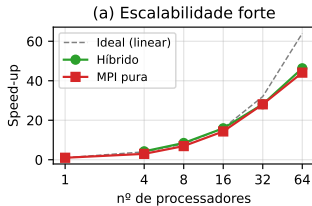
Proposta: versão **híbrida MPI + OpenMP** em dois níveis:

- **Entre máquinas (MPI)** —
Coordenador/Trabalhador: distribui blocos de linhas **sob demanda** (1 processo pesado/máquina).
- **Dentro da máquina (OpenMP)** — *workpool sem mestre*: threads pegam a próxima linha de uma **estrutura de dados** compartilhada.

Carga irregular $\Rightarrow$ **balanceamento dinâmico**. Rodado em **4 máquinas** do Atlantica.



Resultados



Escalabilidade forte

proc.	Híbrido		MPI pura	
	<i>S</i>	<i>E</i>	<i>S</i>	<i>E</i>
4	4,26	1,06	2,97	0,74
8	8,49	1,06	6,92	0,86
16	15,9	0,99	14,3	0,90
32	28,1	0,88	28,1	0,88
64	46,3	0,72	44,1	0,69

Fraca (speed-up)

proc.	Híb.	MPI
4	4,2	3,1
16	16,6	15,5
32	31,5	30,3
64	50,4	48,5

Coordenador (4 nós)

Alocação	Tempo
Dedica 1 núcleo	2,09s
Não dedica	2,48s
Dedica um nó	4,04s

proc. (processadores) = **núcleos** no híbrido (4 trab. × threads) e **processos** na MPI pura. *S* = speed-up (vezes mais rápido). *E* = eficiência ($S \div \text{proc.}$; 1,00 = ideal).

Conclusões

No fim, a versão híbrida cumpriu o objetivo do trabalho e entregou bons resultados.

- Ficou **correta**: segue o modelo proposto (Coordenador/Trabalhador entre as máquinas e *workpool* sem mestre dentro da máquina) e gera exatamente a mesma imagem da versão sequencial.
- **Escalou bem**: chegamos a cerca de $46\times$ de speed-up em 64 núcleos, quase o ideal. A única perda de eficiência é o limite físico do Hyper-Threading, não do nosso algoritmo.
- **Valeu a pena ser híbrido**: mesmo empatando com a MPI pura, ele usa bem menos processos, gastando menos memória e comunicação, o que tende a compensar em escalas maiores.
- E a análise mostrou que, para o coordenador, basta reservar **um núcleo**, nunca uma máquina inteira.

Um bom exemplo prático de como combinar MPI e OpenMP num cluster.